

Problem Set 3 – Multi-Dimensional Packet Scrambler

Kshitiz Reddy
M.Tech AIML, School of AI, Bennett University

Source Code

```
1  """
2  Program: Multi-Dimensional Packet Scrambler
3  Purpose: Runs a data packet through a four-stage pipeline using only
4           built-in list operations, no external libraries.
5  Author: Kshitiz
6  """
7
8  packet = [5, 12, 0, 8, 21, 34, 7, 19, 0, 3]
9
10 # --- Stage 1: Input Validation ---
11 if packet and len(packet) >= 10:
12     print("Validation passed. Processing packet...")
13 else:
14     print("Validation failed: packet is empty or too short.")
15
16 print(f"Initial packet: {packet}")
17
18 # --- Stage 2: Middle-Out Swap ---
19 midpoint = len(packet) // 2
20 front_half = packet[:midpoint]
21 back_half = packet[midpoint:]
22 scrambled = back_half[::-1] + front_half
23
24 print("--- Stage 2: Middle-Out Swap ---")
25 print(f"Scrambled: {scrambled}")
26 print(f"Non-destructive check, id(packet) == id(front_half): "
27       f"{id(packet) == id(front_half)}")
28
29 # --- Stage 3: In-Place Correction ---
30 print("--- Stage 3: In-Place Correction ---")
31
32 middle_index = len(scrambled) // 2
33 if type(scrambled[middle_index]) is int:
34     scrambled.insert(middle_index + 1, "SYNC-BIT")
35 print(f"After Sync-Bit insertion: {scrambled}")
36
37 while 0 in scrambled:
38     scrambled.remove(0)
39 print(f"After zero removal: {scrambled}")
40
41 # --- Stage 4: Memory Integrity Check ---
42 print("--- Stage 4: Memory Integrity Check ---")
43 print(f"Original packet: {packet}")
44 print(f"Final scrambled: {scrambled}")
45
46 first, *middle, last = scrambled
47 print(f"Header: {first} Footer: {last} Body length: {len(middle)}")
48
49 # Note: front_half, back_half, and scrambled are all built through
50 # slicing/concatenation, never direct assignment, so none of them alias
51 # packet. The id() check above confirms slicing returns a new list.
```

```

52 # Since Stage 3's mutations only run on scrambled, packet stays exactly
53 # as it was at Stage 1.
54
55
56 def scramble(data):
57     """Stretch goal - same pipeline, returns result instead of printing."""
58     mid = len(data) // 2
59     front, back = data[:mid], data[mid:]
60     result = back[::-1] + front
61
62     mi = len(result) // 2
63     if result and type(result[mi]) is int:
64         result.insert(mi + 1, "SYNC-BIT")
65
66     while 0 in result:
67         result.remove(0)
68
69     return result

```

Trace Log

```

1 Validation passed. Processing packet...
2 Initial packet: [5, 12, 0, 8, 21, 34, 7, 19, 0, 3]
3 --- Stage 2: Middle-Out Swap ---
4 Scrambled: [3, 0, 19, 7, 34, 5, 12, 0, 8, 21]
5 Non-destructive check, id(packet) == id(front_half): False
6 --- Stage 3: In-Place Correction ---
7 After Sync-Bit insertion: [3, 0, 19, 7, 34, 5, 'SYNC-BIT', 12, 0, 8, 21]
8 After zero removal: [3, 19, 7, 34, 5, 'SYNC-BIT', 12, 8, 21]
9 --- Stage 4: Memory Integrity Check ---
10 Original packet: [5, 12, 0, 8, 21, 34, 7, 19, 0, 3]
11 Final scrambled: [3, 19, 7, 34, 5, 'SYNC-BIT', 12, 8, 21]
12 Header: 3 Footer: 21 Body length: 7

```

Output matches the worked example in the assignment exactly: scrambled list, Sync-Bit insertion, zero removal, and the final Header/Footer/Body-length unpacking.

Submission Notes

Stage 4 reprints packet, untouched since Stage 1, next to the final scrambled. `front_half`, `back_half`, and `scrambled` are all built via slicing or concatenation, never direct assignment, so none alias `packet` – the `id()` check in Stage 2 confirms this. Since Stage 3's mutations only touch `scrambled`, `packet` stays exactly as it was.